Homework 11 Solutions

Prerith Arunkumar (prerith2)

A hand-written K-means algorithm, K-means color compression of an image, and hierarchical clustering of the `mtcars` data.

Use of AI tools. I used Claude (Claude Code) to draft the `R` code and exposition and to run it. Every number below comes from the code output shown in this report, and I checked the numbers quoted in the text against it.

```
library(png)
```

```
Warning: package 'png' was built under R version 4.4.3
```

Question 1: Implementing K-means

(a) One iteration

```
seed <- 432
set.seed(seed)
n <- 10
X <- matrix(rnorm(n * 2), n, 2, dimnames = list(NULL, c("x1", "x2")))
C0 <- sample(rep(1:2, length.out = n))   # 5 per cluster, so neither is empty
C0
```

```
 [1] 1 2 1 1 2 2 2 1 1 2
```

The seed is 432. The initial assignment puts five observations in each cluster.

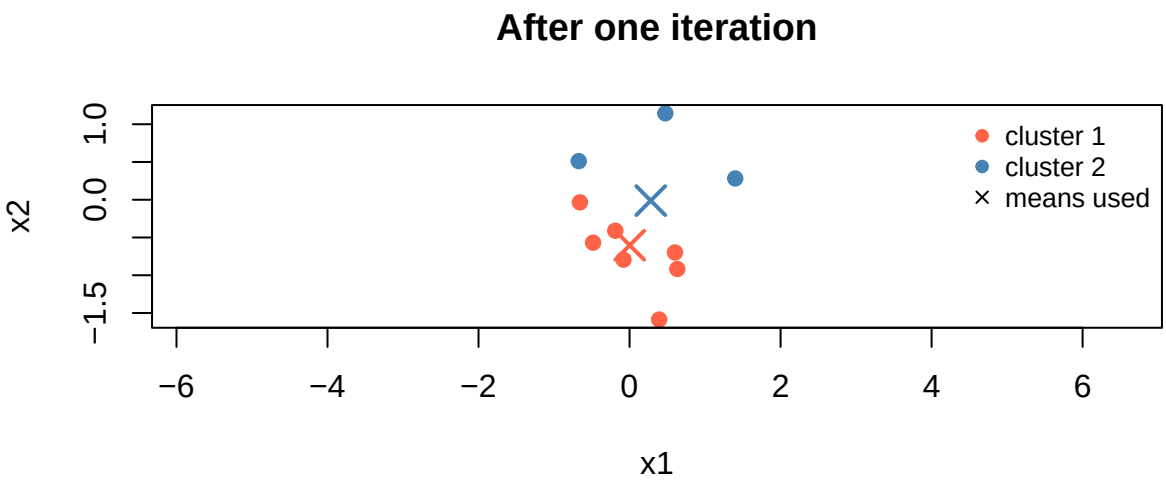
```
m0 <- rbind(colMeans(X[C0 == 1, ]), colMeans(X[C0 == 2, ]))
d0 <- cbind(rowSums(sweep(X, 2, m0[1, ])^2), rowSums(sweep(X, 2, m0[2, ])^2))
C1 <- max.col(-d0, ties.method = "first")
round(m0, 3)
```

```
        x1     x2
[1,] 0.003 -0.602
[2,] 0.281 -0.011
```

```
C1
```

```
 [1] 1 2 1 1 2 1 2 1 1 1
```

```
plot(X, col = c("tomato", "steelblue")[C1], pch = 19, asp = 1,
     main = "After one iteration")
points(m0, col = c("tomato", "steelblue"), pch = 4, cex = 2, lwd = 2)
legend("topright", c("cluster 1", "cluster 2", "means used"), pch = c(19, 19, 4),
       col = c("tomato", "steelblue", "black"), bty = "n", cex = 0.8)
```



The rows of `round(m0, 3)` are the two cluster means computed from the random assignment. `C1` gives the updated assignment: each point goes to the closer of those two means.

(b) The K-means function

```
my_kmeans <- function(X, K, init, max_iter = 100) {
  X <- as.matrix(X)
  C <- init
  centers <- matrix(NA_real_, K, ncol(X))
  update_centers <- function(C, centers) {
    for (k in 1:K) if (any(C == k)) centers[k, ] <- colMeans(X[C == k, , drop = FALSE])
    centers                                   # an empty cluster keeps its previous mean
  }
  converged <- FALSE
  for (it in seq_len(max_iter)) {
    centers <- update_centers(C, centers)
    d <- sapply(1:K, function(k) colSums((t(X) - centers[k, ])^2))   # n x K squared distances
    newC <- max.col(-matrix(d, ncol = K), ties.method = "first")
    if (all(newC == C)) { converged <- TRUE; break }
    C <- newC
  }
  centers <- update_centers(C, centers)       # means from the final assignments
  wss <- sum((X - centers[C, , drop = FALSE])^2)
  list(cluster = C, centers = centers, wss = wss, iter = it, converged = converged)
}
random_init <- function(n, K) sample(rep(1:K, length.out = n))   # every cluster nonempty when
n >= K
```

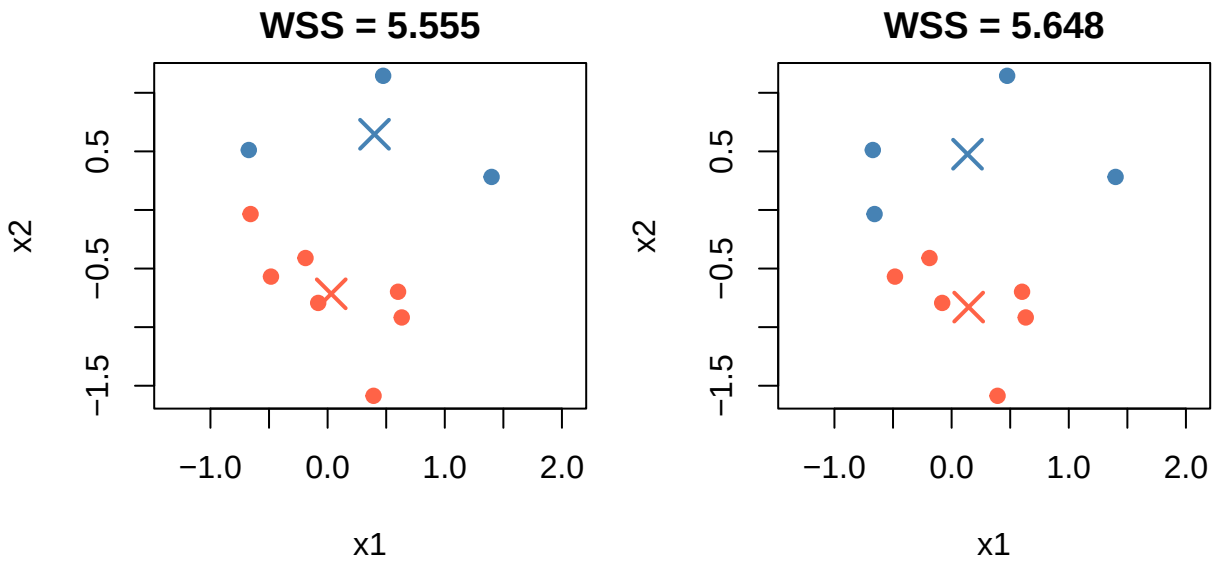
The function works for any number of columns and any K. It stops when an assignment step changes nothing, or after `max_iter` iterations, and reports which happened.

(c) Two random starts

```r
set.seed(1); fitA <- my_kmeans(X, 2, random_init(n, 2))
set.seed(2); fitB <- my_kmeans(X, 2, random_init(n, 2))
for (f in list(fitA, fitB)) {
  cat("clusters:", f$cluster, " WSS =", round(f$wss, 4), " iterations =", f$iter,
      " converged =", f$converged, "\n")
  print(round(f$centers, 3))
}
```

```
clusters: 1 2 1 1 2 1 2 1 1 1  WSS = 5.5546  iterations = 2  converged = TRUE
      [,1]   [,2]
[1,] 0.031 -0.715
[2,] 0.400  0.646
clusters: 2 2 1 1 2 1 2 1 1 1  WSS = 5.6483  iterations = 2  converged = TRUE
      [,1]   [,2]
[1,] 0.146 -0.829
[2,] 0.136  0.476
```

```r
par(mfrow = c(1, 2), mar = c(4, 4, 2, 1))
for (f in list(A = fitA, B = fitB)) {
  plot(X, col = c("tomato", "steelblue")[f$cluster], pch = 19, asp = 1,
       main = sprintf("WSS = %.3f", f$wss))
  points(f$centers, col = c("tomato", "steelblue"), pch = 4, cex = 2, lwd = 2)
}
```




```r
same_grouping <- all(fitA$cluster == fitB$cluster) || all(fitA$cluster == 3 - fitB$cluster)
same_grouping
```

```
[1] FALSE
```

The two runs (seeds 1 and 2 for the initial assignment) both converge, but `same_grouping` is FALSE. The within-cluster sums of squares are 5.555 and 5.648. Each K-means step can only lower or keep the objective. The algorithm stops at the first assignment that no single step can improve, which is a local minimum, not necessarily the global one. With only ten points and no clear gap between groups, several partitions are stable in this sense, and the starting assignment decides which one is reached. Run A reaches the lower objective here. In practice one runs many random starts and keeps the fit with the smallest WSS.

Question 2: Image color compression

(a) The image and its pixel matrix

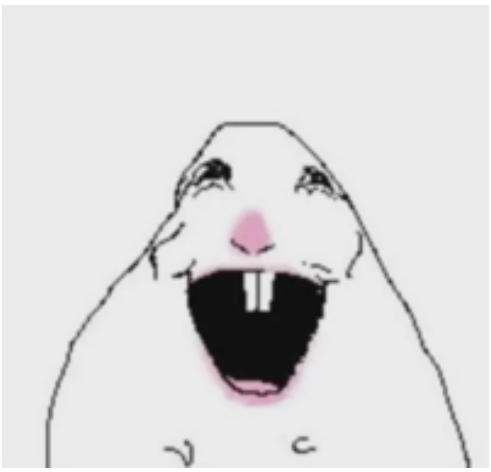
```
img <- readPNG("q2-image.png")[, , 1:3]   # resized to at most 200 x 200 beforehand; drop
alpha if present
dim(img)
```

```
[1] 190 200   3
```

```
P <- matrix(img, ncol = 3, dimnames = list(NULL, c("red", "green", "blue")))
dim(P)
```

```
[1] 38000     3
```

```
show_img <- function(a, main) {
  plot(0:1, 0:1, type = "n", axes = FALSE, xlab = "", ylab = "", main = main, asp = dim(a)[1]
  / dim(a)[2])
  rasterImage(a, 0, 0, 1, 1)
}
par(mar = c(0, 0, 2, 0)); show_img(img, "Original")
```



The image is 190 by 200 pixels with 3 color channels, so the pixel matrix has 38000 rows (one per pixel) and 3 columns (red, green, blue), each on the $[0, 1]$ scale.

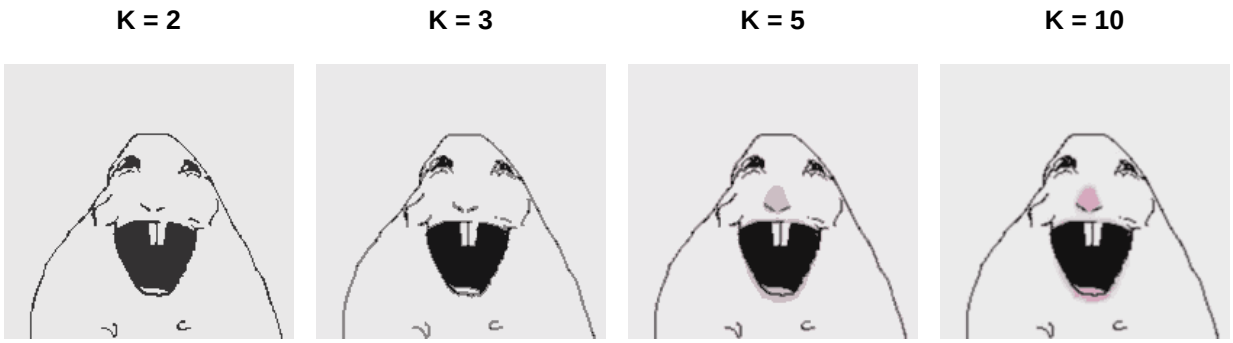
(b) Compression for K = 2, 3, 5, 10

```
Ks <- c(2, 3, 5, 10)
fits <- lapply(Ks, function(K) { set.seed(100 + K); my_kmeans(P, K, random_init(nrow(P), K),
max_iter = 200) })
names(fits) <- paste0("K=", Ks)
```

```r
for (i in seq_along(Ks)) {
  f <- fits[[i]]
  cat(sprintf("K = %d: WSS = %.2f, iterations = %d, converged = %s\n", Ks[i], f$wss, f$iter,
  f$converged))
  print(round(f$centers, 3))
}
```

```
K = 2: WSS = 474.01, iterations = 7, converged = TRUE
      [,1]  [,2]  [,3]
[1,] 0.912 0.910 0.911
[2,] 0.200 0.191 0.195
K = 3: WSS = 137.46, iterations = 13, converged = TRUE
      [,1]  [,2]  [,3]
[1,] 0.917 0.915 0.916
[2,] 0.532 0.516 0.523
[3,] 0.092 0.087 0.089
K = 5: WSS = 38.07, iterations = 27, converged = TRUE
      [,1]  [,2]  [,3]
[1,] 0.386 0.369 0.377
[2,] 0.572 0.559 0.565
[3,] 0.920 0.920 0.920
[4,] 0.801 0.744 0.770
[5,] 0.081 0.077 0.079
K = 10: WSS = 12.02, iterations = 70, converged = TRUE
       [,1]  [,2]  [,3]
 [1,] 0.729 0.721 0.724
 [2,] 0.074 0.070 0.071
 [3,] 0.515 0.501 0.507
 [4,] 0.241 0.221 0.230
 [5,] 0.399 0.381 0.389
 [6,] 0.886 0.870 0.878
 [7,] 0.615 0.605 0.610
 [8,] 0.827 0.797 0.811
 [9,] 0.838 0.663 0.745
[10,] 0.921 0.921 0.921
```

```r
par(mfrow = c(1, 4), mar = c(0, 0, 2, 0))
for (i in seq_along(Ks)) {
  f <- fits[[i]]
  rec <- array(f$centers[f$cluster, ], dim = dim(img))   # every pixel -> its cluster mean,
same [0,1] RGB scale
  show_img(pmin(pmax(rec, 0), 1), sprintf("K = %d", Ks[i]))
}
```



(c) Choosing K

```
wss <- sapply(fits, `[[`, "wss")
round(cbind(WSS = wss, share_of_K2 = wss / wss[1]), 3)
```

```
         WSS share_of_K2
K=2  474.013       1.000
K=3  137.464       0.290
K=5   38.072       0.080
K=10  12.019       0.025
```

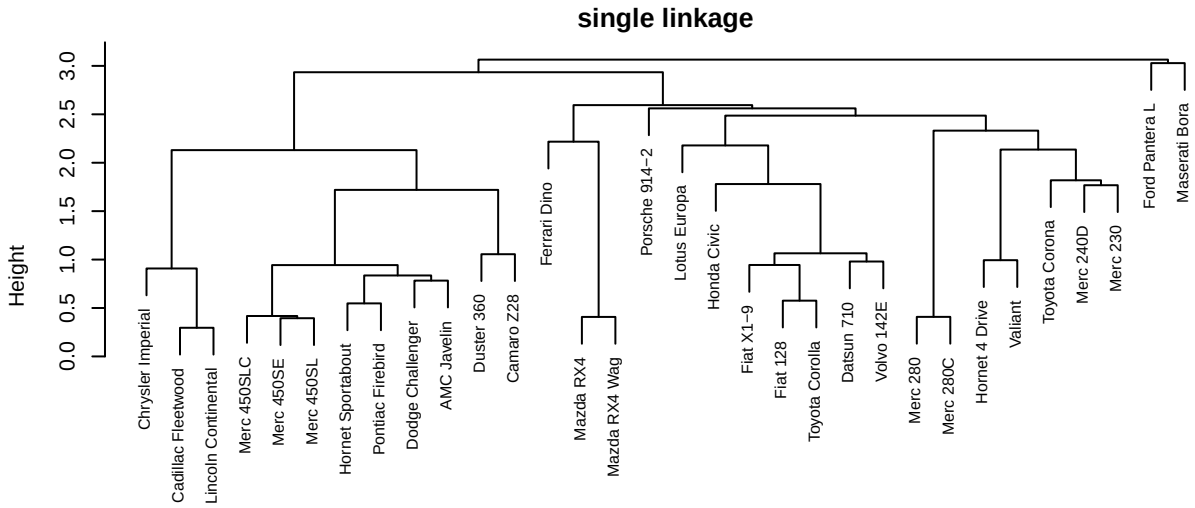
I would choose $K = 5$. The image is a black-and-white line drawing on a light gray background, with a small pink nose and lip area. With $K = 2$ and $K = 3$ every cluster mean is a shade of gray (the three channels are nearly equal in each row of the means above), so the pink disappears completely. With $K = 5$ one cluster mean is pinkish (red channel 0.8 against green 0.74), and the nose and lips come back. The within-cluster sum of squares falls to 8% of its $K = 2$ value. Going to $K = 10$ lowers it further, from 38.1 to 12.0, and gives a stronger pink, but most of the extra clusters are additional grays for the anti-aliased edges of the lines. They add little that is visible in the reconstruction, so the gain is not worth twice as many colors.

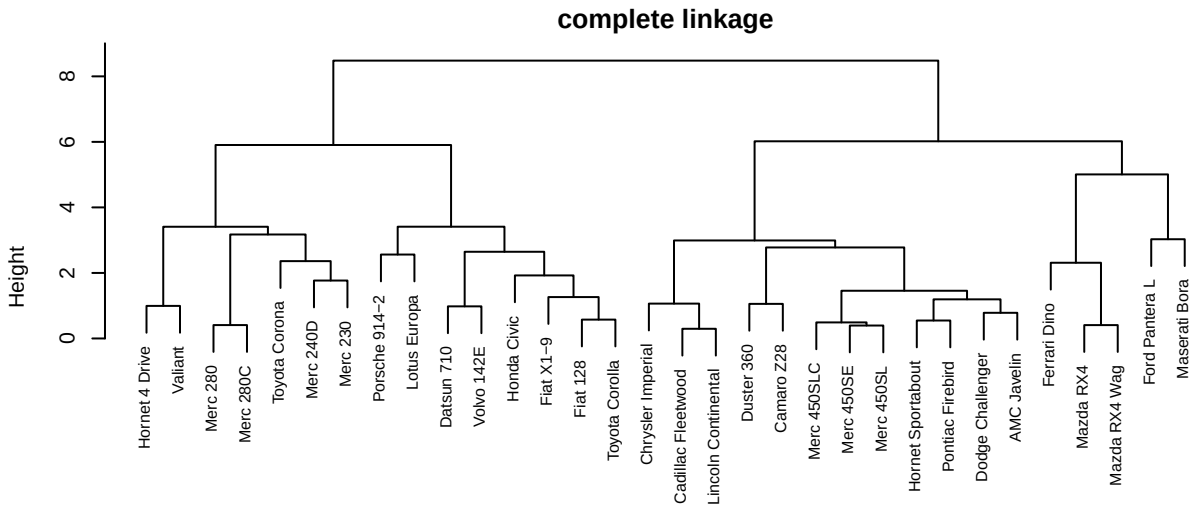
Question 3: Hierarchical clustering of cars

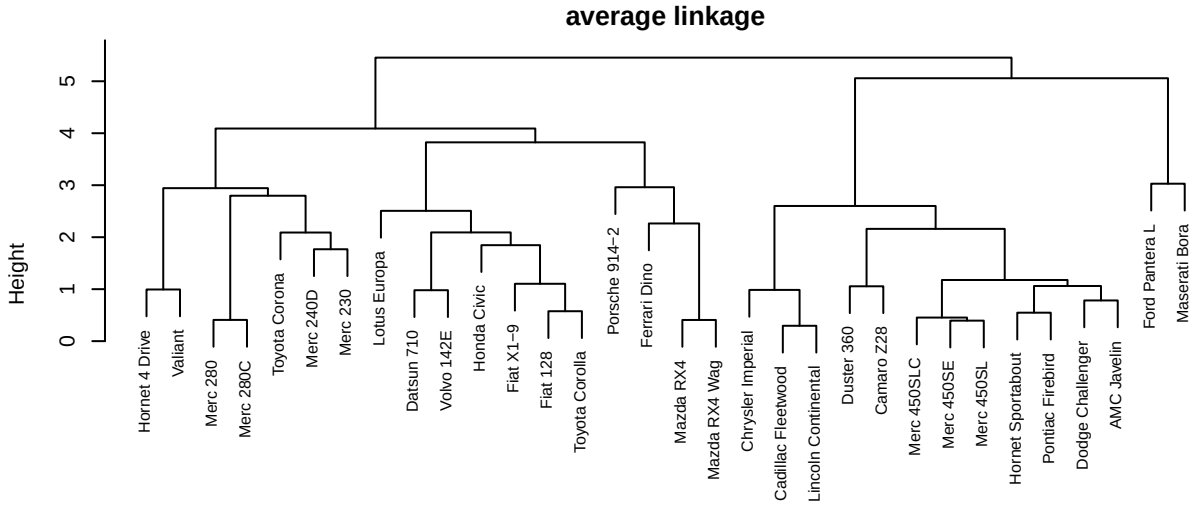
(a) Three linkages

```
cars <- read.csv("data/mtcars.csv")
Z <- scale(cars[, -1])                  # mean 0, sd 1 for each of the 11 variables
rownames(Z) <- cars$car
D <- dist(Z)                            # Euclidean
hcs <- lapply(c(single = "single", complete = "complete", average = "average"),
              function(m) hclust(D, method = m))
```

```
par(mfrow = c(3, 1), mar = c(1, 4, 2, 1))
for (m in names(hcs)) plot(hcs[[m]], main = paste(m, "linkage"), xlab = "", sub = "", cex =
0.75)
```








```
sapply(hcs, function(h) round(range(h$height), 2))   # merge heights: min and max
```

```
     single complete average
[1,]   0.30     0.30    0.30
```

```
[2,]   3.06     8.48    5.45
```

```
sapply(hcs, function(h) table(factor(cutree(h, 3), levels = 1:3)))   # cluster sizes at 3
groups
```

```
  single complete average
1     30        5      18
2      1       15      12
3      1       12       2
```

Single linkage merges at low heights (its last merge is at 3.06, against 8.48 for complete linkage) and chains: cars join a growing cluster one or two at a time, and the three-group cut only peels off the Ford Pantera L and the Maserati Bora as singletons. Average linkage also isolates those two cars, together, before splitting the rest into groups of 18 and 12. Complete linkage gives compact groups of more even size (5, 15, 12) with the largest gaps near the top of the tree. All three methods agree that the Pantera and the Bora are the most unusual cars; they differ in whether such outliers get their own cluster.

(b) A chosen clustering

```
k <- 3
grp <- cutree(hcs$complete, k)
split(names(grp), grp)
```

```
$`1`
[1] "Mazda RX4"      "Mazda RX4 Wag"  "Ford Pantera L" "Ferrari Dino"
[5] "Maserati Bora"

$`2`
 [1] "Datsun 710"     "Hornet 4 Drive" "Valiant"        "Merc 240D"
 [5] "Merc 230"       "Merc 280"       "Merc 280C"      "Fiat 128"
 [9] "Honda Civic"    "Toyota Corolla" "Toyota Corona"  "Fiat X1-9"
[13] "Porsche 914-2"  "Lotus Europa"   "Volvo 142E"

$`3`
 [1] "Hornet Sportabout"   "Duster 360"          "Merc 450SE"
 [4] "Merc 450SL"          "Merc 450SLC"         "Cadillac Fleetwood"
 [7] "Lincoln Continental" "Chrysler Imperial"   "Dodge Challenger"
[10] "AMC Javelin"         "Camaro Z28"          "Pontiac Firebird"
```

```
round(aggregate(cars[, -1], by = list(cluster = grp), FUN = mean), 2)
```

```
  cluster   mpg  cyl   disp     hp drat   wt  qsec   vs   am gear carb
1       1 18.50 6.80 223.40 198.80 3.84 3.00 15.62 0.00 1.00 4.60 5.20
2       2 24.65 4.53 131.65  91.33 3.90 2.58 19.16 0.93 0.53 3.93 1.80
3       3 15.05 8.00 357.62 194.17 3.12 4.10 17.14 0.00 0.00 3.00 3.08
```

```
round(rev(tail(hcs$complete$height, 4)), 2)   # heights of the last four merges
```

```
[1] 8.48 6.02 5.90 5.01
```

I use complete linkage with 3 clusters. The largest gap in the dendrogram is below the final merge, so two clusters is the strongest split by height alone; the next two merges are at similar heights, so the tree does not clearly favor three over four. I chose three because the two-cluster cut merges two groups that the table shows are quite different, while the fourth cut only divides the 15-car economy group into groups of 8 and 7. The groups are:

- **Cluster 1 (5 cars: the two Mazda RX4s, Ford Pantera L, Ferrari Dino, Maserati Bora): high-performance manual cars.** All are manual with V-shaped engines (`am` = 1, `vs` = 0), with the highest horsepower (about 199), the most carburetors and gears, and the fastest quarter-mile time, at moderate weight.
- **Cluster 2 (15 cars, such as the Honda Civic, Toyota Corolla, Fiat 128 and Mercedes 240D/280): light economy cars.** They have the best fuel economy (about 24.7 mpg), the fewest cylinders, about 91 hp, the lowest weight, and mostly straight engines.
- **Cluster 3 (12 cars, such as the Cadillac Fleetwood, Lincoln Continental, Camaro Z28 and Mercedes 450 series): heavy V8 automatics.** All have 8 cylinders and automatic transmissions, the largest displacement and weight, and the worst fuel economy (about 15.1 mpg).

Clusters 1 and 3 both have high horsepower, which is why the two-cluster cut joins them, but they differ in transmission, weight, gears and carburetors.

Why standardize. The variables are on very different scales: displacement is in the hundreds of cubic inches and horsepower in the hundreds, while weight is a few thousand pounds recorded in units of 1000 lb, and `vs` and `am` are 0/1. Euclidean distance on the raw values would be dominated by `disp` and `hp`, so the clustering would essentially be a clustering on engine size alone. Standardizing gives each variable the same spread, so each contributes comparably to the distance.